



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

Name: Sumit kr. Mahato PRN: 25070122195

ASSIGNMENT No : 2

TITLE : Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Assignment 2: Methods and Static Members

Problem Statement

Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Learning Objectives

To understand and implement default, parameterized, and copy constructors for object initialization.

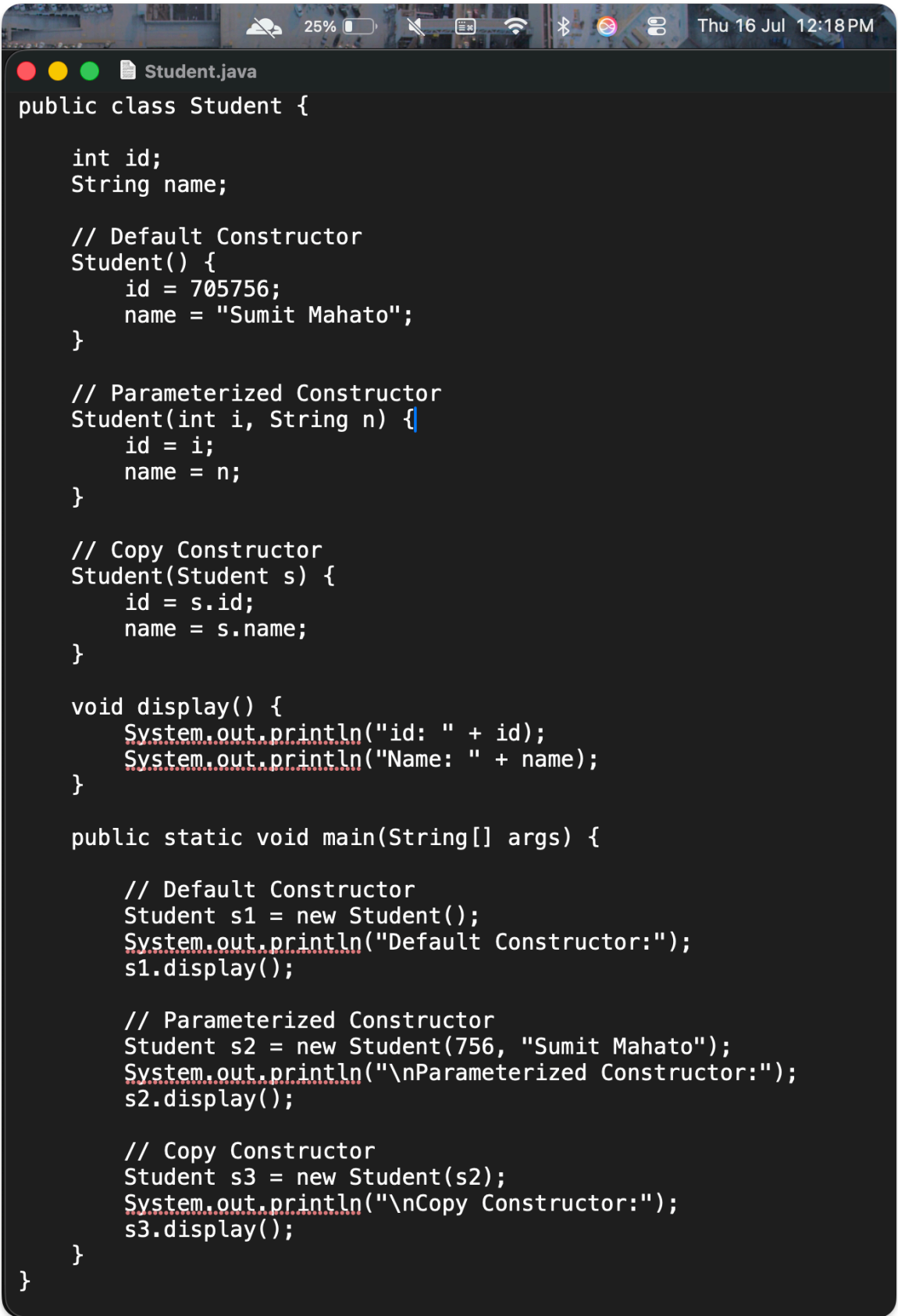
Theory

Constructors are special member functions that are automatically invoked when an object is created. Java supports default constructors provided by the compiler, parameterized constructors for initializing objects with user-defined values, and copy constructors (implemented manually) for creating duplicate objects. Constructors improve code readability and simplify object initialization while supporting object-oriented programming principles.

Exercises

1. Create a **Student class** using default and parameterized constructors to initialize student name and roll number.

Source code:



```
Student.java
public class Student {

    int id;
    String name;

    // Default Constructor
    Student() {
        id = 705756;
        name = "Sumit Mahato";
    }

    // Parameterized Constructor
    Student(int i, String n) {
        id = i;
        name = n;
    }

    // Copy Constructor
    Student(Student s) {
        id = s.id;
        name = s.name;
    }

    void display() {
        System.out.println("id: " + id);
        System.out.println("Name: " + name);
    }

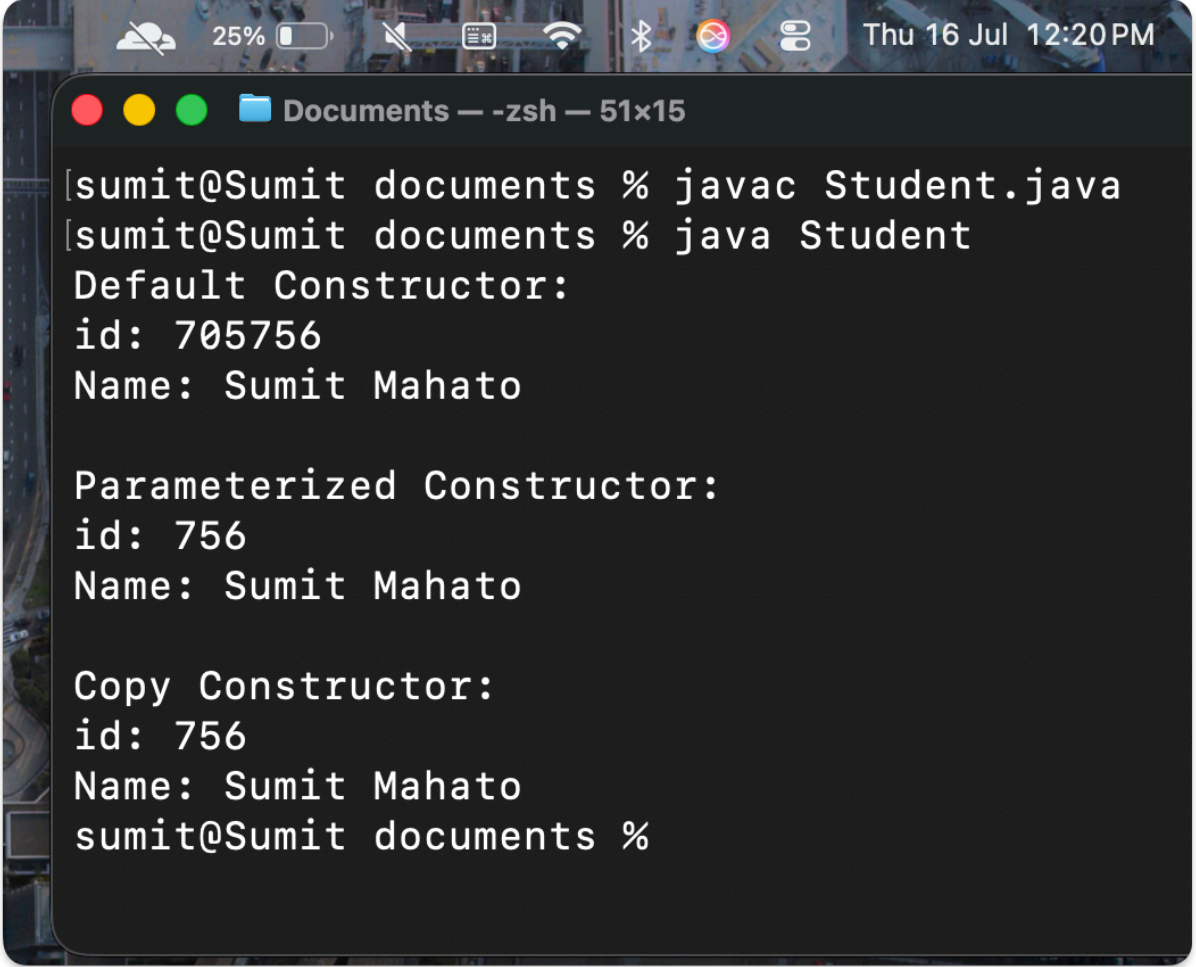
    public static void main(String[] args) {

        // Default Constructor
        Student s1 = new Student();
        System.out.println("Default Constructor:");
        s1.display();

        // Parameterized Constructor
        Student s2 = new Student(756, "Sumit Mahato");
        System.out.println("\nParameterized Constructor:");
        s2.display();

        // Copy Constructor
        Student s3 = new Student(s2);
        System.out.println("\nCopy Constructor:");
        s3.display();
    }
}
```

Output:

A screenshot of a macOS terminal window. The title bar shows standard macOS window controls (red, yellow, green buttons) and the text 'Documents — -zsh — 51x15'. The terminal content shows the compilation of 'Student.java' and the execution of 'java Student'. The output displays three constructors: a Default Constructor, a Parameterized Constructor, and a Copy Constructor, each showing an 'id' and a 'Name'. The terminal ends with a prompt for the next command.

```
[sumit@Sumit documents % javac Student.java  
[sumit@Sumit documents % java Student  
Default Constructor:  
id: 705756  
Name: Sumit Mahato  
  
Parameterized Constructor:  
id: 756  
Name: Sumit Mahato  
  
Copy Constructor:  
id: 756  
Name: Sumit Mahato  
sumit@Sumit documents %
```

2. Develop a **Mobile Phone Inventory System** using different constructors to initialize mobile details and create duplicate object records.

Source code:

```
Mobile.java
public class Mobile {

    String brand;
    double price;

    // Default Constructor
    Mobile() {
        brand = "Samsung";
        price = 25000;
    }

    // Parameterized Constructor
    Mobile(String b, double p) {
        brand = b;
        price = p;
    }

    // Copy Constructor
    Mobile(Mobile m) {
        brand = m.brand;
        price = m.price;
    }

    void display() {
        System.out.println("Brand : " + brand);
        System.out.println("Price : " + price);
    }

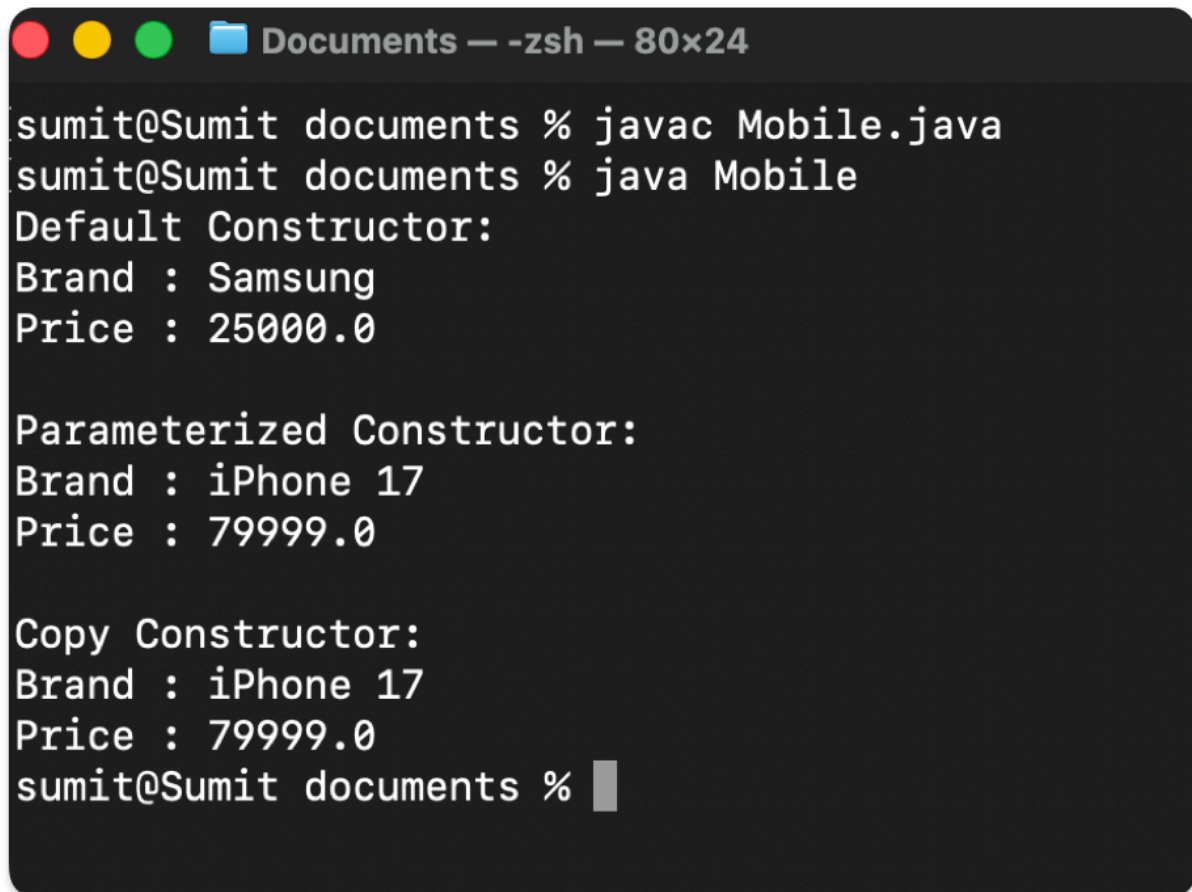
    public static void main(String[] args) {

        // Default Constructor
        Mobile m1 = new Mobile();
        System.out.println("Default Constructor:");
        m1.display();

        // Parameterized Constructor
        Mobile m2 = new Mobile("iPhone 17", 79999);
        System.out.println("\nParameterized Constructor:");
        m2.display();

        // Copy Constructor
        Mobile m3 = new Mobile(m2);
        System.out.println("\nCopy Constructor:");
        m3.display();
    }
}
```

Output:

A terminal window titled "Documents — -zsh — 80x24" with standard macOS window controls (red, yellow, green buttons). The terminal shows the execution of Java code. It starts with "sumit@Sumit documents % javac Mobile.java" and "sumit@Sumit documents % java Mobile". The output displays three constructor types: "Default Constructor:" with "Brand : Samsung" and "Price : 25000.0"; "Parameterized Constructor:" with "Brand : iPhone 17" and "Price : 79999.0"; and "Copy Constructor:" with "Brand : iPhone 17" and "Price : 79999.0". The prompt "sumit@Sumit documents %" is followed by a cursor bar.

```
sumit@Sumit documents % javac Mobile.java
sumit@Sumit documents % java Mobile
Default Constructor:
Brand : Samsung
Price : 25000.0

Parameterized Constructor:
Brand : iPhone 17
Price : 79999.0

Copy Constructor:
Brand : iPhone 17
Price : 79999.0
sumit@Sumit documents %
```

Conclusion

This program demonstrates how Java constructors are used for object initialization in different scenarios. The default constructor assigns fixed/standard values when no data is provided, the parameterized constructor allows initializing an object with specific values at creation time, and the copy constructor (implemented manually) duplicates the data of an existing object into a new one. This shows how constructors support flexible and efficient object creation, which is essential in real-world data management systems.